SNIPER — Especificação final do indicador *unlock_pressure_rank*

Arquitetura final para observado automatizado, reconstruído automatizado e proxy estrutural, com escopo histórico limitado ao “cemitério de ativos” e a uma amostra controlada de blue chips.

Projeto	Base metodológica	Data	Escopo
SNIPER	UPS da v10.10 + governança de dados point-in-time	2026-03-08	Implementação de produção e pesquisa

Decisão executiva

- 1) O live daqui para frente será baseado em observado automatizado com snapshots diários.
- 2) A reconstrução histórica automatizada será limitada aos ativos críticos do cemitério e a um conjunto pequeno de blue chips âncora.
- 3) A cauda longa sem cronograma confiável ficará coberta por um proxy estrutural contínuo, nunca por scraping frágil promovido indevidamente a dado observado.

1. Objetivo e critério de projeto

Este documento define a arquitetura final para gerar e persistir o indicador unlock_pressure_rank sem dependência de API paga, preservando a lógica econômica do UPS do SNIPER e evitando fraude estatística no histórico.

- Preservar a lógica original do UPS: unlocks previstos em 30 dias, ponderados por bucket econômico, com rank cross-sectional no universo elegível.
- Separar explicitamente três naturezas de informação: dado observado automatizado, dado reconstruído automatizado e proxy estrutural.
- Restringir a reconstrução histórica de alto custo ao cemitério de ativos obrigatório e a uma amostra pequena de blue chips âncora; a cauda longa ficará no proxy.
- Persistir tudo com rastreabilidade point-in-time para que backtest, CPCV, C2ST e produção enxerguem exatamente o que estava disponível em cada data.

2. Requisitos herdados do SNIPER v10.10

A implementação deve continuar fiel ao documento-base do SNIPER:

- o universo é point-in-time e inclui obrigatoriamente ativos colapsados;
- o UPS filtra o universo com base na pressão de unlock em 30 dias;
- os buckets canônicos e pesos herdados são Team/Founders = 1.5, VC/Investors = 1.2, Ecosystem = 0.8 e Airdrop/Public = 0.6;
- o filtro operacional permanece no top 25% de pressão (equivalente a corte acima do percentil 75 do raw score).

3. Escopo e política de ROI de engenharia

Não faz sentido reconstruir a internet inteira para milhares de tokens obscuros. O motor de reconstrução histórica com Wayback CDX e parser determinístico será deliberadamente restrito a dois grupos.

Grupo	Escopo	Tratamento
Cemitério obrigatório	LUNA, FTT, CEL, LUNA2 e demais colapsados explicitamente mandatórios no universo	Obrigatório para impedir survivorship bias e backtest ilusório
Blue chips âncora	Amostra alvo de 10–15 nomes, configurável. Sugestão inicial: BTC, ETH, BNB, SOL, XRP, ADA, DOGE, AVAX, LINK, DOT, TRX, MATIC/POL	Ancorar distribuição, QA e comparação do proxy ao longo do tempo
Cauda longa	Demais ativos elegíveis do universo	Sem reconstrução histórica pesada; usar proxy estrutural quando não houver observado

4. Arquitetura-alvo do fator

A implementação deve materializar três camadas de feature e um estado obrigatório de governança. Nenhuma camada substitui silenciosamente a outra.

Campo	Origem	Função
unlock_pressure_rank_observed	Cronograma vindo de fonte estruturada e automatizada, capturado em snapshot point-in-time	Feature observada de maior prioridade
unlock_pressure_rank_reconstructed	Cronograma reconstruído automaticamente a partir de evidência histórica pública, com confiança alta	Feature histórica automatizada de segunda prioridade
unlock_overhang_proxy_rank_full	Proxy estrutural completo, calculado quando houver circ e total_supply point-in-time válidos	Fallback econômico principal quando o schedule não existir
unlock_fragility_proxy_rank_fallback	Proxy degradado de fragilidade de liquidez para histórico/longa cauda quando supply history completo não estiver disponível	Fallback operacional de último nível, separado do proxy full
unlock_feature_state	Enum de auditoria: OBSERVED / RECONSTRUCTED / PROXY_FULL / PROXY_FALLBACK / MISSING	Rastreabilidade obrigatória para auditoria, drift/C2ST e serving; fora do vetor X preditivo

Opcionalmente pode-se gerar um campo de compatibilidade para o pipeline legado:

```
Campo opcional de reporting/serving (não usar no treino):
unlock_pressure_rank_selected_for_reporting(i,t) =
  observed,        se unlock_pressure_rank_observed disponível e quality = ok
  reconstructed,   senão se unlock_pressure_rank_reconstructed disponível e confidence >= 0.85
  proxy_full,      senão se unlock_overhang_proxy_rank_full disponível
  proxy_fallback,  senão se unlock_fragility_proxy_rank_fallback disponível
  NaN,             caso contrário
```

Esse campo existe apenas para dashboards, triagem operacional e exports. Os componentes originais permanecem persistidos separadamente e são os únicos candidatos ao vetor X.

5. Definições formais e fórmulas

5.1 Buckets canônicos e pesos

Bucket	Peso	Exemplos de labels
Team/Founders	1.5	team, founders, core contributors, advisors, contributor sale
VC/Investors	1.2	seed, private sale, strategic sale, investors, backers
Ecosystem	0.8	foundation, treasury, ecosystem, grants, rewards, incentives
Airdrop/Public	0.6	airdrop, public sale, IDO/IEO, launchpool, testnet incentive
Other	diagnóstico somente	label não mapeado automaticamente

O bucket Other não entra no cálculo final do raw score por padrão. Ele é apenas diagnóstico. Se unknown_bucket_ratio > 15%, o score observado/reconstruído não é promovido ao modelo sem revisão.

5.2 Score observado ou reconstruído

```
Para cada ativo i e data t:

events_30d(i,t,c) = soma dos tokens com unlock em (t, t + 30 dias] mapeados ao bucket c
circ(i,t)         = circulating_supply point-in-time válida na data t
weights           = {Team/Founders:1.5, VC/Investors:1.2, Ecosystem:0.8, Airdrop/Public:0.6}

ups_raw(i,t) = Σ_c [ weights[c] * events_30d(i,t,c) / max(circ(i,t), ε) ]

unlock_pressure_rank(i,t) = percent_rank( winsorize_cs(ups_raw(i,t)) )
                            dentro do universo elegível na data t

Filtro operacional do universo:
ativo é excluído se ups_raw(i,t) > percentil_75_cs(ups_raw(t))
```

5.3 Proxy estrutural contínuo

O proxy não tenta adivinhar datas exatas de unlock. Ele mede overhang estrutural de oferta travada, concentração econômica e fragilidade de liquidez.

Modo FULL (somente quando circ e total_supply point-in-time forem válidos):

vested_overhang(i,t) = max(total_supply(i,t) - circulating_supply(i,t), 0) / max(circulating_supply(i,t), ε)

insider_share_pti(i,t) = (alloc_team_founders + alloc_vc_investors) / max(alloc_total_known, ε)
usando apenas distribuição point-in-time observada ou reconstruída; é proibido retropropagar distribuição atual

```
para o passado.

liq_penalty(i,t) = sqrt( market_cap(i,t) / max(avg_30d_volume(i,t), ε) )

proxy_full_raw(i,t) = log(1 + vested_overhang(i,t)) * (1 + insider_share_pti(i,t)) * liq_penalty(i,t)

unlock_overhang_proxy_rank_full(i,t) =
   percent_rank( winsorize_cs(proxy_full_raw(i,t)) ) dentro do universo elegível na data t

Modo FALLBACK (quando total_supply histórico ou supply history completo não estiver disponível):

insider_share_pti_lite(i,t) = insider_share point-in-time se observado/reconstruído; caso contrário 0.0
Para a cauda longa histórica pré-snapshots sem reconstrução de allocation, insider_share_pti_lite = 0.0.

proxy_fallback_raw(i,t) = sqrt( market_cap(i,t) / max(avg_30d_volume(i,t), ε) ) * (1 + insider_share_pti_lite(i,t))

unlock_fragility_proxy_rank_fallback(i,t) =
   percent_rank( winsorize_cs(proxy_fallback_raw(i,t)) ) dentro do universo elegível na data t
```

Padronização recomendada: winsorização cross-sectional de 1%/99%, clipping de insider_share para [0, 1], e avg_30d_volume calculado com média móvel dos últimos 30 dias úteis disponíveis.

5.4 Tratamento de empates no percent_rank

A implementação não pode inventar ordenação entre ativos economicamente indistinguíveis. Quando vários ativos compartilham o mesmo valor bruto, todos devem receber exatamente o mesmo percent_rank, de forma determinística e reprodutível.

Regras operacionais obrigatórias:

- Calcular o rank transversalmente por data, sempre no universo elegível point-in-time e apenas entre observações não nulas.
- Usar estritamente rank_avg com tratamento de empates pelo método average.
- A fórmula oficial é: percent_rank(i,t) = (rank_avg(i,t) - 1) / (N_t - 1), onde N_t é o número de ativos elegíveis não nulos na data t.
- Se N_t = 1, definir percent_rank = 0.5 por convenção operacional.
- NaN permanece NaN e não entra no ranking cross-sectional.
- Valores brutos iguais a zero, quando válidos, participam normalmente do ranking e podem formar blocos grandes de empate no piso da distribuição.
- Métodos first, random, dense e max são proibidos para o rank final do fator, pois introduzem ordem artificial ou deslocam blocos empatados para posições não neutras.

6. Fontes e prioridade por camada

Camada	Fonte primária	Campos de interesse

Camada	Fonte primária	Campos de interesse
Observado automatizado	Mobula metadata / multi-metadata	release_schedule, distribution, circulating_supply, total_supply, market_cap, volume
Validação de mercado	CoinGecko coins/markets e market_chart / market_chart/range	price, market_cap, total_volume, circulating_supply atual; histórico dedicado de circulating/total supply somente quando o plano/chave permitir
Validação secundária de unlocks	DefiLlama Unlocks	próximo evento, valor do unlock, calendário, export .csv quando exposto
Reconstrução histórica	Wayback CDX + páginas oficiais arquivadas	capturas históricas de tokenomics, unlocks, vesting, whitepaper e docs

A ordem de promoção é sempre: observado estruturado > reconstruído com confiança alta > proxy. Scraping textual frágil sem validação não é promovido para observado nem para reconstruído.

7. Observado automatizado — especificação de produção

A camada observada é a base do live daqui para frente. Ela deve ser completamente automatizada e snapshotada todos os dias.

1. Mapear ativo do universo para identifiers externos: symbol, contract_address, id CoinGecko, asset Mobula.
2. Executar coleta diária de metadata da Mobula por ativo e gravar o payload bruto integral com timestamp de coleta.
3. Extrair release_schedule e distribution; normalizar labels para os buckets canônicos; rejeitar labels ambíguos para Other.
4. Executar coleta diária de mercado no CoinGecko para price, market_cap, total_volume e circulating_supply; persistir payload bruto integral.
5. Calcular ups_raw_observed apenas se houver release_schedule válido, circ > 0 e unknown_bucket_ratio <= 15%.
6. Calcular unlock_pressure_rank_observed de forma cross-sectional no universo elegível da data.
7. Persistir resultado, estado, diagnósticos, payloads brutos, hashes de conteúdo e timestamps de snapshot.

8. Reconstruído automatizado — motor histórico limitado e determinístico

A reconstrução histórica é restrita ao cemitério e ao conjunto pequeno de blue chips âncora. O objetivo não é reconstituir toda a internet; é cobrir de forma auditável os casos que mais distorcem o backtest.

8.1 Camada A — fontes estruturadas prioritárias

- Tentar primeiro qualquer resposta estruturada atualmente disponível: Mobula, páginas/CSV de DefiLlama Unlocks e, quando aplicável, endpoints/documentos técnicos diretamente ligados ao protocolo.
- Se o ativo tiver schedule estruturado atual e o problema for apenas histórico, usar essa estrutura para aprender o template e o vocabulário de buckets.
- Capturar sempre o payload bruto, a origem, o horário de coleta e um hash do conteúdo.

8.2 Camada B — descoberta e recuperação histórica via Wayback CDX

A descoberta deve ser automatizada, com escopo controlado, e sem depender de curadoria humana diária.

1. Descobrir URLs candidatas a partir do CoinGecko /coins/{id}: website(s), docs, GitHub, whitepaper e links oficiais relacionados.
2. Gerar heurísticas de URL para páginas como /tokenomics, /vesting, /unlocks, /docs/tokenomics, /whitepaper, /litepaper, /blog.
3. Manter allowlist apenas para domínios oficiais do projeto e documentos diretamente linkados no site oficial; não rastrear blogs de terceiros como fonte primária.
4. Consultar o Wayback CDX para cada URL candidata e obter capturas em ou antes da data as_of.
5. Selecionar a captura mais recente <= as_of; se houver múltiplas capturas no mesmo dia, preferir HTTP 200, mime type estável e menor distância temporal para as_of.
6. Baixar o HTML/PDF arquivado e gravar snapshot bruto com metadados de origem e fingerprint.

8.3 Camada C — parser determinístico e registrador de templates

A extração histórica não pode ser feita por LLM livre. Ela deve ser determinística, reproduzível e restrita a formatos conhecidos.

- Formatos aceitos para promoção: JSON embutido, CSV, tabela HTML, tabela Markdown, PDF tabular simples.
- Formatos aceitos apenas como evidência auxiliar: texto corrido, posts longos de blog, screenshots e páginas sem estrutura.
- Cada domínio/template terá um parser versionado. Exemplo: parser_whitepaper_pdf_v1, parser_docs_html_table_v2.
- Se o conteúdo vier sem estrutura confiável ou o parser falhar, o item não é promovido para reconstructed; ele fica apenas como evidência bruta arquivada.

8.4 Camada D — validação quantitativa antes de promoção

Regra	Teste	Ação
V1	Tokens do evento >= 0 e datas válidas	Rejeitar evento inválido
V2	Soma dos eventos não pode exceder supply plausível do ativo	Rejeitar snapshot
V3	Buckets mapeados aos canônicos ou Other	Marcar quality = review se Other > 15%
V4	Coerência temporal: captura <= as_of e unlock em janela futura relativa a as_of	Rejeitar se houver lookahead
V5	Concordância cruzada: duas fontes batem OU uma fonte oficial bate com uma agregadora	Elevar confiança
V6	Coerência com supply posterior, quando disponível	Aumentar ou reduzir confiança

8.5 Camada E — score de confiança da reconstrução

```
reconstruction_confidence =
    0.35 * source_score
  + 0.20 * parse_score
  + 0.20 * cross_source_agreement
  + 0.15 * supply_consistency
  + 0.10 * time_consistency

Onde:
source_score           = 1.00 se fonte oficial do projeto / 0.75 se agregadora respeitável /
0.50 se apenas arquivo histórico auxiliar
parse_score            = 1.00 se JSON/CSV/tabela bem estruturada / 0.70 se PDF tabular simples /
```

```
0.30 se semi-estruturado
cross_source_agreement = 1.00 se há confirmação independente / 0.50 se apenas uma fonte boa /
0.00 se conflito aberto
supply_consistency     = 1.00 se consistente com supply/float posteriores / 0.50 se inconclusivo
/ 0.00 se inconsistente
time_consistency       = 1.00 se a captura é claramente <= as_of / 0.00 caso contrário

Promoção:
confidence >= 0.85  -> unlock_pressure_rank_reconstructed elegível
0.70 a <0.85        -> persistir apenas como evidência; não promover ao modelo
< 0.70              -> descartar para feature, manter payload bruto
```

9. Política de consumo no modelo e campo opcional de reporting

O sistema deve separar rigorosamente quatro coisas: persistência de dados, auditoria/governança, consumo pelo modelo preditivo e conveniência de reporting. unlock_feature_state, reconstruction_confidence, quality_flag e o campo opcional de reporting não entram no vetor X preditivo; servem para auditoria, monitoramento de cobertura, C2ST/drift, ablação e serving operacional. O modelo deve consumir apenas colunas contínuas ortogonais, com NaN quando a camada específica não existir.

Contexto	Regra
Persistir sempre	unlock_pressure_rank_observed, unlock_pressure_rank_reconstructed, unlock_overhang_proxy_rank_full, unlock_fragility_proxy_rank_fallback, unlock_feature_state, quality flags e diagnósticos
Campo de compatibilidade/serving	unlock_pressure_rank_selected_for_reporting (opcional), nunca no vetor X
Treino recomendado	usar colunas ortogonais separadas: unlock_pressure_rank_observed, unlock_pressure_rank_reconstructed, unlock_overhang_proxy_rank_full e unlock_fragility_proxy_rank_fallback; preencher com NaN quando a camada não existir. unlock_feature_state, confidence e quality ficam fora do vetor X e servem só para auditoria/monitoramento.
Produção	o modelo consome as mesmas colunas ortogonais do treino; unlock_pressure_rank_selected_for_reporting serve apenas para dashboards, triagem e integrações downstream. Proibido usar qualquer informação posterior a t.

10. Descoberta automática de URLs oficiais

Para evitar manutenção manual diária, a descoberta de fontes do motor histórico deve partir do próprio ecossistema oficial do ativo.

1. Consultar CoinGecko /coins/{id} para obter website principal, repositórios e links oficiais.
2. Extrair do HTML do site oficial links para whitepaper, litepaper, docs, tokenomics, blog e governance.
3. Aplicar heurísticas sobre o domínio oficial: /tokenomics, /unlocks, /vesting, /docs/tokenomics, /whitepaper, /litepaper.
4. Gerar um registry versionado por ativo com campos: url, url_type, domain, is_official, first_seen_ts, last_seen_ts.
5. Para cada data histórica consultada, usar apenas URLs cuja descoberta seja justificável por informação disponível até aquela data, ou URLs estáveis do domínio oficial.

11. Persistência — schema mínimo obrigatório

11.1 Tabelas brutas

Tabela	Campos mínimos
raw_mobula_snapshots	snapshot_ts, asset_id, payload_json, payload_hash, http_status
raw_coingecko_snapshots	snapshot_ts, cg_id, payload_json, payload_hash, http_status
raw_wayback_fetches	as_of_date, asset_id, source_url, capture_ts, mime_type, payload_path, payload_hash
raw_defillama_unlocks	snapshot_ts, asset_id, page_type, extracted_json, payload_hash

11.2 Tabelas normalizadas

Tabela	Campos mínimos
asset_source_registry	asset_id, url, url_type, domain, is_official, source_priority, active_flag
unlock_events_normalized	asset_id, as_of_date, event_date, raw_label, bucket, tokens, source_type, source_url, confidence
feature_unlock_daily	date, asset_id, ups_raw_observed, unlock_pressure_rank_observed, ups_raw_reconstructed, unlock_pressure_rank_reconstructed, proxy_full_raw, unlock_overhang_proxy_rank_full, proxy_fallback_raw, unlock_fragility_proxy_rank_fallback, unlock_pressure_rank_selected_for_reporting, unlock_feature_state, unknown_bucket_ratio, quality_flag, reconstruction_confidence
feature_unlock_diagnostics	date, asset_id, circ, total_supply, insider_share_pti, insider_share_mode, avg_30d_volume, market_cap, source_primary, snapshot_ts, supply_history_mode

12. Regras de qualidade e fallback

- Se release_schedule estiver ausente no live, unlock_pressure_rank_observed = NaN; unlock_feature_state registra OBSERVED/RECONSTRUCTED/PROXY_FULL/PROXY_FALLBACK/MISSING apenas para auditoria e serving, não para o vetor X preditivo.
- Se circulating_supply <= 0 ou nula, não promover observado/reconstruído; manter apenas diagnóstico.
- Se unknown_bucket_ratio > 15%, manter quality_flag = review_required e não promover a camada observada/reconstruída.
- Se houver divergência material entre Mobula e CoinGecko em circ, total_supply ou market_cap, persistir ambas as versões, marcar conflict_flag e não mascarar a divergência.
- Se o motor histórico não encontrar captura válida <= as_of, não há reconstrução; o ativo recai para proxy_full quando houver supply point-in-time suficiente, ou para proxy_fallback quando não houver.
- Nunca preencher manualmente rank cross-sectional. Se houver reconstrução, ela deve gerar eventos brutos; o rank é recalculado no universo inteiro da data.

13. Pipeline operacional

13.1 Pipeline diário de produção

1. Atualizar universo point-in-time do SNIPER para a data t.
2. Coletar snapshots Mobula e CoinGecko para todos os ativos elegíveis.
3. Calcular camada observada quando possível.
4. Calcular proxy estrutural FULL quando houver circ e total_supply point-in-time válidos; caso contrário, calcular apenas o proxy FALLBACK de fragilidade de liquidez.
5. Persistir as camadas separadas; opcionalmente materializar unlock_pressure_rank_selected_for_reporting para dashboards, triagem e exports, nunca para treino/CFI.
6. Persistir features, diagnósticos, payloads brutos e hashes.
7. Executar alertas de cobertura, conflicts, unknown_bucket_ratio e drift de distribuição.

13.2 Pipeline histórico de reconstrução

1. Executar apenas para o cemitério e para a amostra de blue chips âncora.
2. Para cada ativo e data as_of do backtest, descobrir URLs oficiais válidas e consultar capturas históricas via Wayback CDX.
3. Baixar a melhor captura <= as_of, parsear deterministamente, normalizar eventos, validar e pontuar confiança.
4. Se confidence >= 0.85, materializar unlock_events_normalized e calcular ups_raw_reconstructed.
5. Recalcular unlock_pressure_rank_reconstructed de forma cross-sectional no universo daquela data.
6. Persistir tudo com provenance e hashes.

14. Critérios de aceite para integração ao treino

Fase	Entregável	Critério de aprovação
Fase 1	Camada observada live	Cobertura >= 80% do universo live com quality ok ou review explicável; snapshots diários íntegros
Fase 2	Proxy estrutural contínuo (full + fallback)	Cobertura histórica >= 95% do universo elegível com pelo menos uma das duas camadas de proxy quando existirem dados mínimos de mercado
Fase 3	Motor histórico limitado	Cemitério + blue chips com taxa de promoção reconstructed >= 70% dos casos-alvo ou justificativa documentada
Fase 4	Validação de utilidade	Sem lookahead, sem quebra de schema, estabilidade em CPCV/DSR/PBO e prova de que metadados de governança ficaram fora do vetor X
Fase 5	Integração final	Sem lookahead, sem quebra de schema, estabilidade em CPCV/DSR/PBO e documentação de produção concluída

15. Protocolo de pesquisa e validação do fator

- Executar event study antes de qualquer celebração de feature importance: decil 1 versus decil 10 para proxy_full, proxy_fallback e, separadamente, para observado/reconstruído.
- Controlar por size e liquidez nas análises, porque o proxy incorpora market_cap e volume.
- Testar incrementalmente: baseline SNIPER; baseline + proxy_full/proxy_fallback; baseline + proxy_full/proxy_fallback + observado/reconstruído.

- Aplicar a mesma disciplina temporal do restante do sistema: purging, embargo, CPCV, e avaliação por DSR/PBO quando houver múltiplas tentativas.
- Manter shadow mode da camada observada até haver histórico suficiente de snapshots próprios. Metadados de governança permanecem fora do vetor X, mas devem ser monitorados em paralelo para cobertura, quebras de regime e integridade do dado.

16. Pseudocódigo de referência

```
def build_unlock_features(date_t, universe):
    rows = []

    observed_inputs = collect_live_snapshots(date_t, universe)        # Mobula + CoinGecko
    proxy_inputs    = collect_proxy_inputs(date_t, universe)          # market_cap + avg_30d_volume +
supply/alocação quando disponíveis

    for asset in universe:
        obs      = compute_observed(asset, date_t, observed_inputs.get(asset))
        rec      = load_or_build_reconstructed(asset, date_t)         # somente cemetery + anchors
        prx_full = compute_proxy_full(asset, date_t, proxy_inputs.get(asset))
        prx_fb   = compute_proxy_fallback(asset, date_t, proxy_inputs.get(asset))

        state = "MISSING"
        selected_for_reporting = None

        if obs.is_valid:
            selected_for_reporting = obs.rank
            state = "OBSERVED"
        elif rec.is_valid and rec.confidence >= 0.85:
            selected_for_reporting = rec.rank
            state = "RECONSTRUCTED"
        elif prx_full.is_valid:
            selected_for_reporting = prx_full.rank
            state = "PROXY_FULL"
        elif prx_fb.is_valid:
            selected_for_reporting = prx_fb.rank
            state = "PROXY_FALLBACK"

        rows.append({
            "asset": asset,
            "ups_raw_observed": obs.raw,
            "unlock_pressure_rank_observed": obs.rank,
            "ups_raw_reconstructed": rec.raw,
            "unlock_pressure_rank_reconstructed": rec.rank,
            "proxy_full_raw": prx_full.raw,
            "unlock_overhang_proxy_rank_full": prx_full.rank,
            "proxy_fallback_raw": prx_fb.raw,
```

```
        "unlock_fragility_proxy_rank_fallback": prx_fb.rank,
        "unlock_pressure_rank_selected_for_reporting": selected_for_reporting,
        "unlock_feature_state": state,
        "reconstruction_confidence": rec.confidence,
        "unknown_bucket_ratio": max(obs.unknown_ratio, rec.unknown_ratio),
        "quality_flag": choose_quality(obs, rec, prx_full, prx_fb)
    })

  return rank_cross_sectional_and_persist(rows, date_t)

# Vetor X do modelo: apenas as colunas contínuas ortogonais.
# unlock_feature_state e selected_for_reporting não entram no treino.
```

17. Pseudocódigo do motor histórico

```
def build_reconstructed_for_asset(asset, as_of_date):
    candidate_urls = discover_official_urls(asset)                    # CoinGecko + heuristics
    captures = []

    for url in candidate_urls:
        captures += wayback_cdx_query(url=url, latest_before=as_of_date)

    best = choose_best_capture(captures, as_of_date)
    if not best:
        return Reconstructed.invalid("no_capture")

    payload = fetch_wayback_capture(best)
    parsed  = deterministic_parse(payload, source_url=best.url)
    if not parsed.is_structured:
        return Reconstructed.invalid("unstructured_payload")

    normalized = normalize_unlock_events(parsed.events, as_of_date)
    checks = validate_reconstruction(normalized, asset, as_of_date)
    conf = score_reconstruction(best, parsed, checks)

    if conf < 0.85:
        persist_evidence_only(asset, as_of_date, best, parsed, conf)
        return Reconstructed.invalid("confidence_below_threshold")

    raw = compute_ups_raw_from_events(normalized, as_of_date, asset)
    rank = None   # rank final é calculado depois, no universo cross-sectional da data
    persist_reconstructed(asset, as_of_date, best, normalized, raw, conf)
    return Reconstructed.valid(raw=raw, rank=rank, confidence=conf)
```

18. Riscos reconhecidos e resposta de engenharia

Risco	Resposta
Layouts quebrados e PDFs ruins	Parser determinístico por template + promoção só com score alto
Falta de snapshots históricos na cauda longa	Não reconstruir; usar proxy estrutural
Confusão entre observed e proxy	Campos separados + state obrigatório

Risco	Resposta
Lookahead involuntário na reconstrução	Captura sempre <= as_of; validação temporal explícita
Custo excessivo de engenharia	Escopo histórico restrito ao cemitério e anchors

19. Referências operacionais

Internas

- SNIPER v10.10 — Especificação Técnica Definitiva.
- SNIPER — Especificação do indicador unlock_pressure_rank (versão anterior).

Externas

- Mobula API — Get Token Unlocks Data: https://docs.mobula.io/guides/token-unlock
- Mobula API — Get Token Vesting and Unlocks Details via API: https://docs.mobula.io/guides/query-token-vesting-and-unlocks-details
- CoinGecko API — Coins List with Market Data: https://docs.coingecko.com/reference/coins-markets
- CoinGecko API — Coin Historical Chart Data by ID / range: https://docs.coingecko.com/reference/coins-id-market-chart e https://docs.coingecko.com/reference/coins-id-market-chart-range
- CoinGecko API — Historical circulating/total supply endpoints (documentados como rotas Pro/Enterprise): https://docs.coingecko.com/reference/coins-id-circulating-supply-chart
- Internet Archive — Wayback APIs / CDX Server: https://archive.org/help/wayback_api.php
- DefiLlama Unlocks: https://defillama.com/unlocks
- Pandas Documentation — Series.rank (semântica de empates e métodos de ranking): https://pandas.pydata.org/docs/reference/api/pandas.Series.rank.html

LightGBM Documentation — Missing Value Handle / categorical features: https://lightgbm.readthedocs.io/en/latest/Advanced-Topics.html